

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

DA3 CAD PROJECT REPORT

Campus Lost-and-Found System Using AWS Cloud Resources

Submitted by

Student Name	Bavisetti Vivek
Registration No.	24BCE1104
Course / Subject	DA3 CAD
Slot	E2+TE2
Submitted To	Preetha Evangeline D
Institution	Vellore Institute of Technology, Chennai
Academic Year	2026-2027

Table of Contents

1. Abstract 3
2. Introduction 3
3. Objectives 3
4. System Architecture 4
5. AWS Services Used 4
6. Application Features 5
7. Implementation Details 6
7.1 Application Home Page 6
7.2 Lost Item Details and Image Display 7
7.3 Item Status Updated to Resolved 7
8. AWS Infrastructure Screenshots 8
8.1 EC2 Instance – Running Status 8
8.2 EC2 Instance Configuration Details 9
8.3 EC2 Security Group Inbound Rules 10
8.4 Amazon S3 – Bucket Overview 11
8.5 Amazon S3 – Uploaded Item Image 12
8.6 Amazon S3 – Encryption & Security Settings 13
8.7 Amazon DynamoDB – Table Configuration 14
8.8 Amazon DynamoDB – Item Record 14
8.9 IAM Role and Permissions 15
8.10 IAM Role – Trust Relationship 15
8.11 CloudWatch – Logging Configuration 16
8.12 CloudWatch – Metrics and Monitoring 16
9. Test Cases and Results 17
10. Conclusion 19
11. References 20

1. Abstract

This report presents the design, implementation, and cloud deployment of a Campus Lost-and-Found System developed as the DA3 CAD project at Vellore Institute of Technology, Chennai. The system is a Python Flask web application hosted on Amazon EC2, with item records stored in Amazon DynamoDB and item photographs stored securely in Amazon S3 using AES-256 server-side encryption. Images are served exclusively through time-limited presigned URLs, ensuring the S3 bucket remains private at all times. Access to AWS services is granted through an IAM instance profile attached to the EC2 instance, eliminating the need for hardcoded credentials. Application events and errors are captured in Amazon CloudWatch Logs for real-time monitoring. The application enables students to report lost or found items, upload photographs, search and filter reports, and update item status, providing a practical and scalable alternative to traditional campus notice boards.

2. Introduction

Managing lost and found items on a university campus is a persistent challenge. Traditional approaches — physical notice boards, manual registers at security desks, or informal social media posts — are fragmented, unreliable, and inaccessible to the wider campus community. Items reported on a notice board may never reach the person who lost them; images cannot be attached; and there is no way to track whether an item has been claimed.

This project addresses this gap by building a cloud-native web application that centralises all lost-and-found activity in one place. Students can submit detailed reports with photographs, browse all existing reports, filter by category or status, and mark items as resolved once they have been returned. The application is fully hosted on AWS, demonstrating how cloud services can be applied to solve everyday institutional problems.

3. Objectives

- Build a responsive web application (HTML, CSS, JavaScript, Python Flask) for reporting, searching, and tracking lost and found campus items.
- Host on Amazon EC2 (Amazon Linux 2023, t3.micro) using Gunicorn and Nginx on port 80 for production-grade serving.
- Store item photographs in Amazon S3 (AES-256 encryption, Block Public Access ON) and serve via time-limited presigned URLs.
- Persist all item records in Amazon DynamoDB (table: CampusLostFoundItems, PK: item_id, PAY_PER_REQUEST billing).
- Grant EC2 access to AWS resources through an IAM instance profile (DA3CampusLostFoundEC2Role) — no hardcoded credentials.
- Capture application events in Amazon CloudWatch Logs (/campus-laf/app) for real-time monitoring and audit.
- Apply VPC and Security Group rules (HTTP 80 open; SSH 22 admin-only) to enforce network-level least privilege.

4. System Architecture

The application follows a three-tier cloud architecture. The presentation layer consists of HTML/CSS/JavaScript pages served through Nginx (port 80) and Flask running on an EC2 instance. The application layer is handled by Flask, which processes form submissions, validates inputs, and communicates with AWS services using the boto3 SDK. The data layer is composed of Amazon DynamoDB for structured item records and Amazon S3 for binary image storage.

Nginx acts as a reverse proxy, forwarding all external HTTP traffic to Gunicorn listening on the internal port 5000. The EC2 instance carries an IAM instance profile so boto3 can obtain temporary credentials automatically from the instance metadata service (IMDS) — no access keys are stored on disk. Images are stored with AES-256 server-side encryption; when a page needs to display an image, Flask generates a presigned GetObject URL valid for one hour, allowing the browser to fetch the image directly from S3 without any public bucket access.

5. AWS Services Used

AWS Service	Resource	Purpose
Amazon EC2	t3.micro, Amazon Linux 2023	Hosts Flask/Gunicorn behind Nginx; public-facing on port 80
Amazon S3	campus-lostfound-24bce1104-2026	Stores item images privately; served via 1-hour presigned URLs
Amazon DynamoDB	CampusLostFoundItems	Stores all item records; item_id PK; PAY_PER_REQUEST billing
AWS IAM	DA3CampusLostFoundEC2Role	Instance profile: least-privilege S3, DynamoDB, CloudWatch access
Amazon CloudWatch	/campus-laf/app log group	Runtime events, errors, health-check pings; 7-day retention
Amazon VPC	Default VPC + Subnet	Network isolation for the EC2 instance
Security Group	DA3-Campus-LostFound-SG	HTTP 80 from 0.0.0.0/0; SSH 22 from admin IP only

6. Application Features

6.1 Report Submission

Users complete a form capturing: item type (lost/found), title, category, description, location, contact details, reporter name, and an optional photograph. JavaScript validates file type (PNG, JPG, GIF, WEBP) and size (max 5 MB) client-side; Flask repeats all validation server-side before writing any data.

6.2 Image Upload to Amazon S3

On form submission with a photograph, Flask calls boto3's upload_fileobj with ServerSideEncryption: AES256 in ExtraArgs. The S3 object key (items/<uuid>.<ext>) is stored in the DynamoDB record. When displaying the item, Flask calls generate_presigned_url to produce a URL valid for one hour, so images are rendered in the browser without any public S3 access.

6.3 Record Storage in DynamoDB

Each report is stored as a DynamoDB item with attributes: item_id (UUID, PK), item_type, title, description, location, contact, category, reporter, s3_key, status (Open/Claimed/Resolved/Expired), created_at, and updated_at. DynamoDB on-demand billing means no provisioned-capacity charges.

6.4 Search and Filter

The Search page accepts any combination of: a free-text keyword matched against title, description, and location; item type; category (Electronics, Keys, Books, etc.); and status. Filtering is performed server-side in Python after a DynamoDB Scan, keeping the query logic simple and maintainable.

6.5 Status Updates

Each item detail page provides a status dropdown. Submission triggers a DynamoDB UpdateItem call that atomically changes the status field and refreshes updated_at. A flash message confirms the change; CloudWatch logs the event for audit purposes.

6.6 Health Check Endpoint

The /api/health endpoint performs live HeadBucket (S3) and DescribeTable (DynamoDB) calls and returns a JSON response confirming connectivity and IAM permissions. HTTP 200 indicates all services are reachable; HTTP 503 signals a degraded state.

7. Implementation Details

7.1 Application Home Page

The home page renders a hero section with quick-report buttons, a statistics row (lost, found, resolved, total — all drawn live from DynamoDB), a keyword search bar, and a grid of the twelve most recent reports. Each card displays the item title, category, location, a presigned-URL thumbnail, and a colour-coded status badge. The layout is fully responsive for mobile viewports.

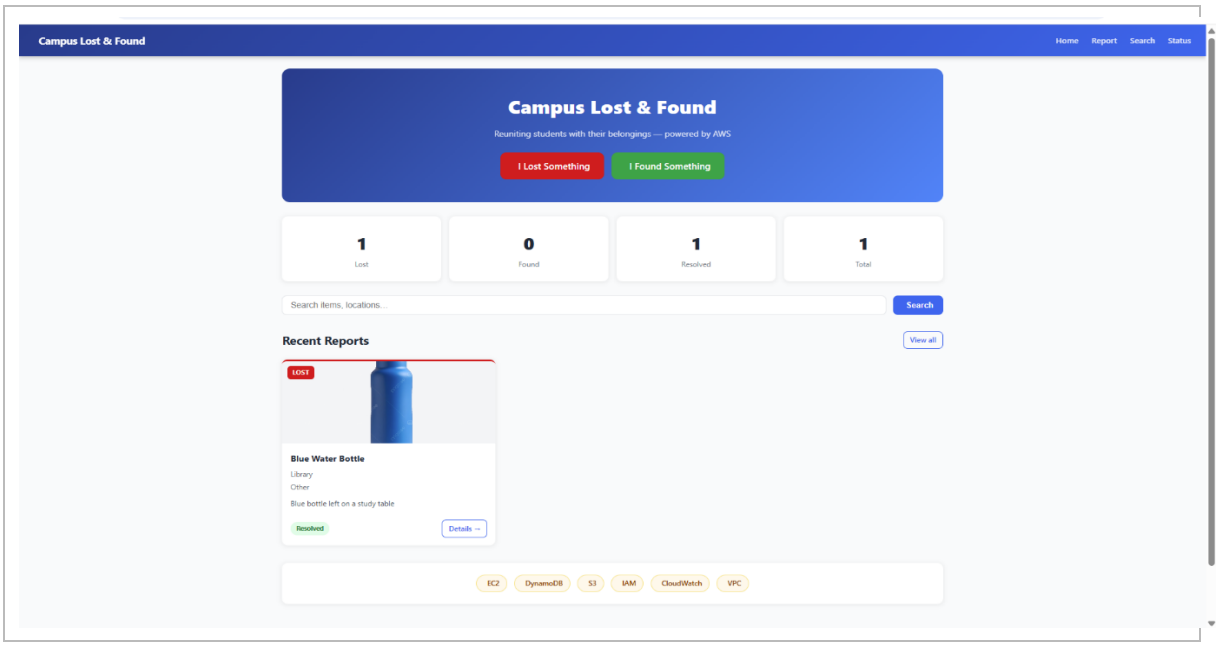


Figure 7.1 — Home page: live DynamoDB statistics and recent item cards with presigned S3 thumbnails

7.2 Lost Item Details and Image Display

The detail page fetches the full DynamoDB record via GetItem and displays all attributes alongside the item photograph. The image is loaded through a one-hour presigned URL generated by Flask — the S3 bucket's Block Public Access setting is never bypassed. A note on the page confirms the image is "Stored privately in AWS S3," and the S3 object key is visible in the record metadata.

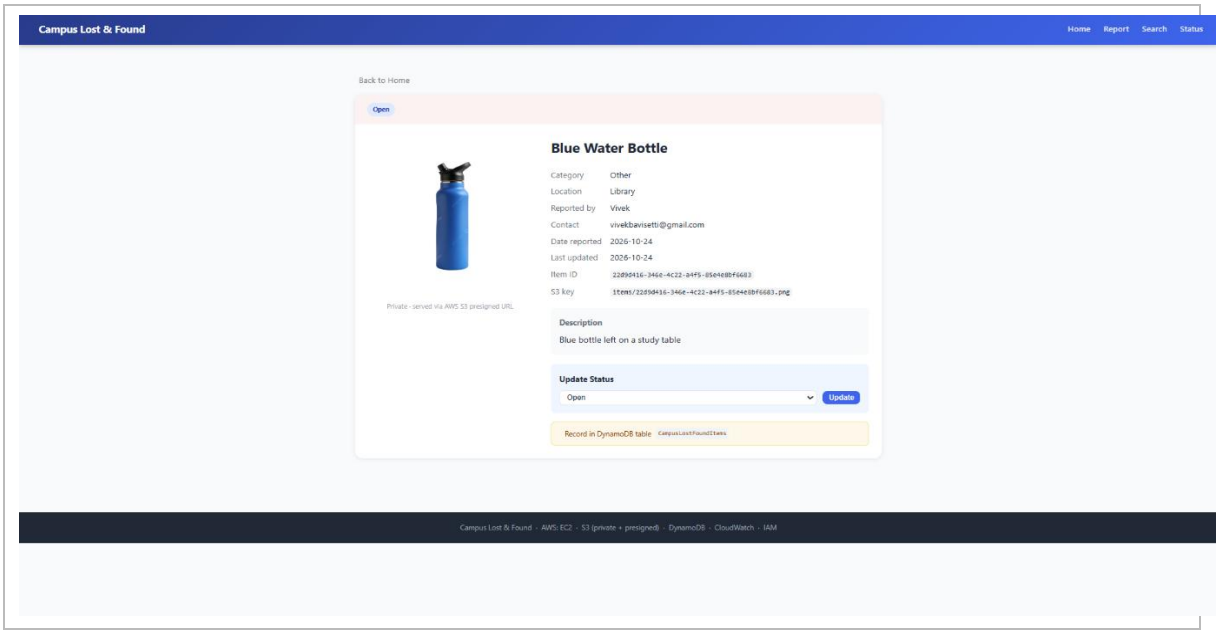


Figure 7.2 — Item detail: full DynamoDB record with all attributes and image served via S3 presigned URL

7.3 Item Status Updated to Resolved

Selecting "Resolved" from the status dropdown and submitting triggers a DynamoDB UpdateItem expression that changes the status field and refreshes updated_at atomically. The page reloads with a green "Resolved" status badge and a success flash message. CloudWatch also records this status-change event in the /campus-laf/app log group.

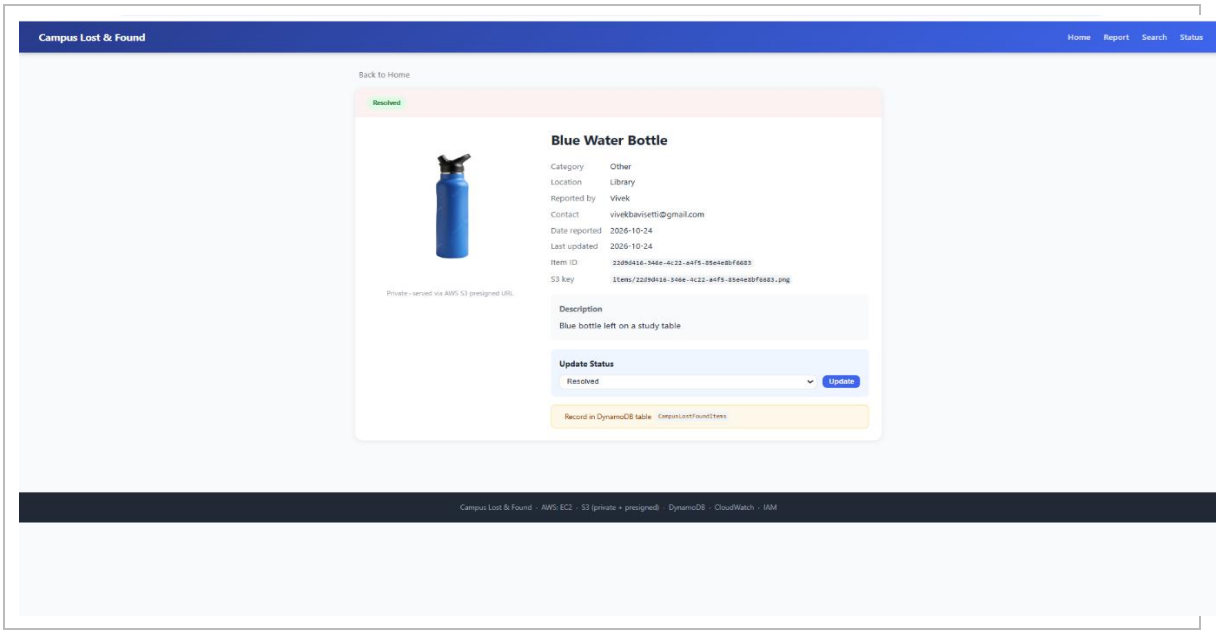


Figure 7.3 — Status updated to "Resolved": DynamoDB record updated; flash confirmation displayed

8. AWS Infrastructure Screenshots

The following screenshots capture the AWS Management Console views of each cloud resource used in this project. They provide evidence that real AWS infrastructure — not simulated or mocked services — underpins the application at every layer.

8.1 EC2 Instance – Running Status

The EC2 instance (Amazon Linux 2023, t3.micro, 8 GiB EBS storage) is in the Running state with a public IPv4 address assigned. Nginx receives all external HTTP traffic on port 80 and reverse-proxies it to Gunicorn on the internal port 5000. The DA3CampusLostFoundEC2Role IAM instance profile is visible in the instance summary, confirming that boto3 can authenticate with AWS services using temporary IMDS credentials without any stored access keys.

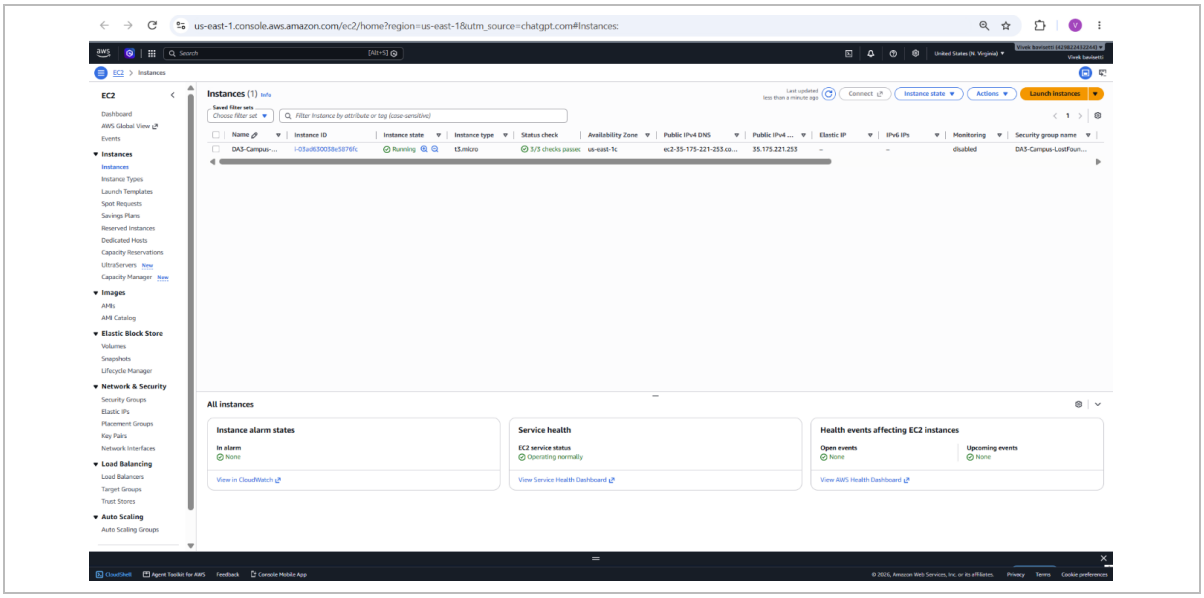


Figure 8.1 — EC2 Console: instance in Running state with public IPv4 address and IAM instance profile attached

8.2 EC2 Instance Configuration Details

The instance detail panel shows the instance type (t3.micro), the assigned VPC and public subnet, the availability zone, the attached IAM instance profile (DA3CampusLostFoundEC2Role), the root EBS volume (8 GiB gp3), and the associated Security Group (DA3-Campus-LostFound-SG). The t3.micro instance type is within the AWS Free Tier 750 hours per month limit, ensuring no EC2 compute charges are incurred during the project evaluation period.

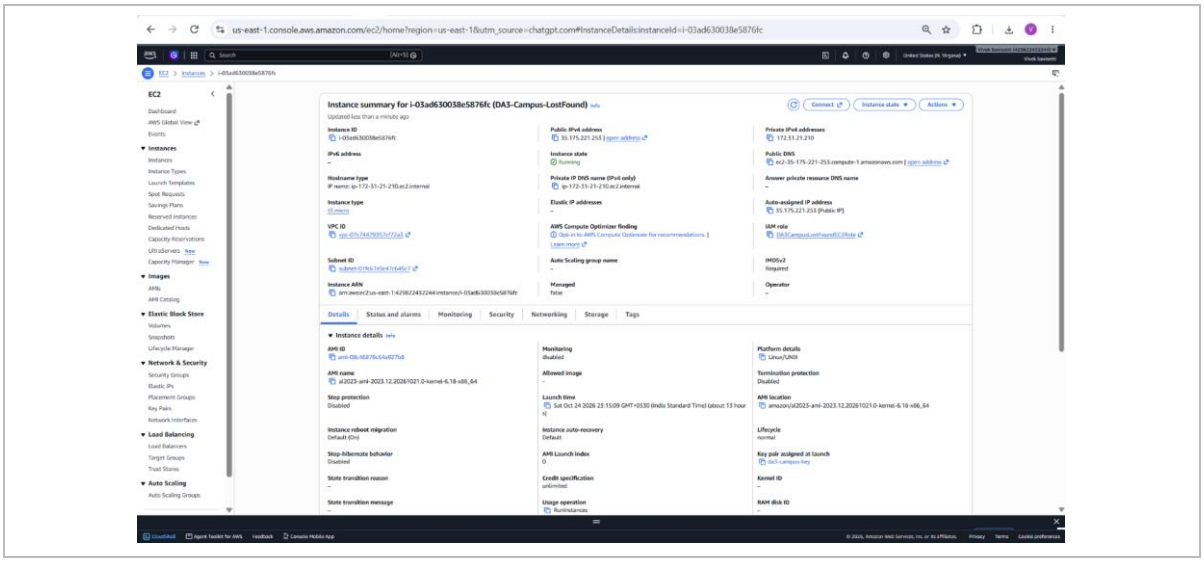


Figure 8.2 — EC2 configuration: instance type, VPC, subnet, IAM profile, Security Group, and EBS volume

8.3 EC2 Security Group – Inbound Rules

The Security Group DA3-Campus-LostFound-SG enforces least privilege at the network level. An inbound rule permits HTTP traffic on port 80 from any IPv4 address (0.0.0.0/0), enabling student access from anywhere on campus or remotely. SSH access on port 22 is restricted to the administrator's specific IP address, preventing unauthorised remote login attempts. All other inbound traffic is implicitly denied by the Security Group's default-deny policy.

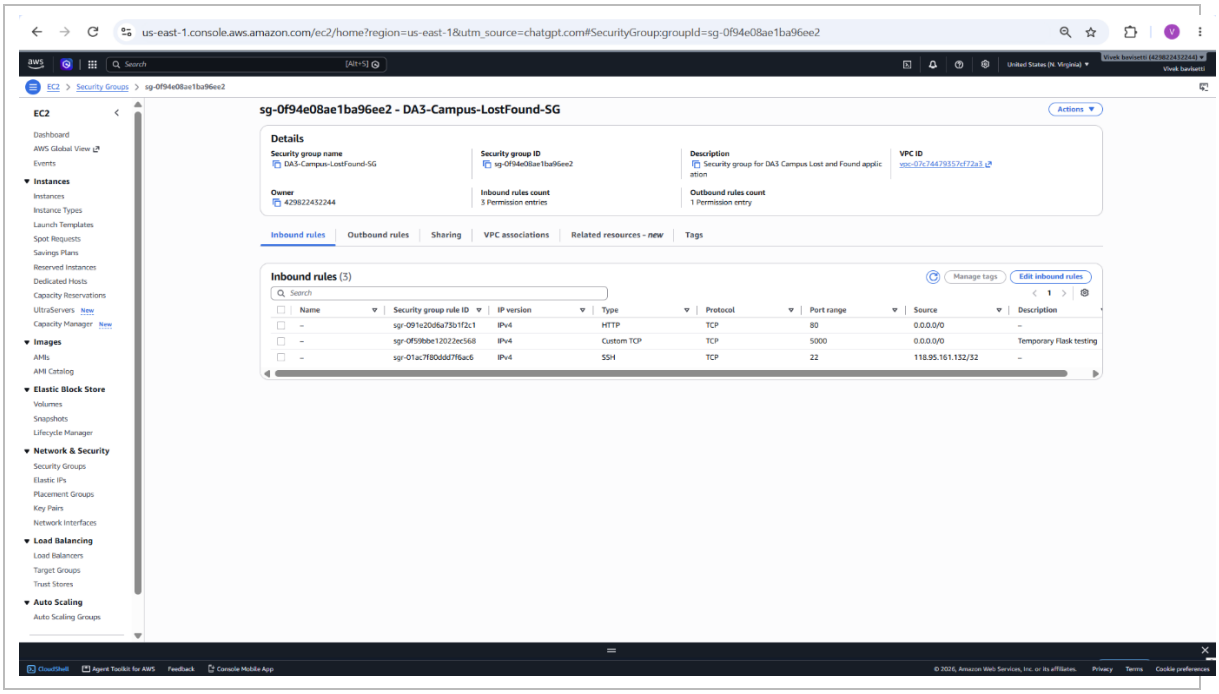


Figure 8.3 — Security Group DA3-Campus-LostFound-SG: HTTP (80) open; SSH (22) restricted to admin IP

8.4 Amazon S3 – Bucket Overview

The S3 bucket campus-lostfound-24bce1104-2026 stores all item photographs uploaded through the application, organised under the items/ prefix. The bucket is located in us-east-1 and is configured with Block Public Access fully enabled, meaning no object can be made publicly readable through bucket policies or object ACLs. All images are accessible exclusively through time-limited presigned URLs generated by the Flask application.

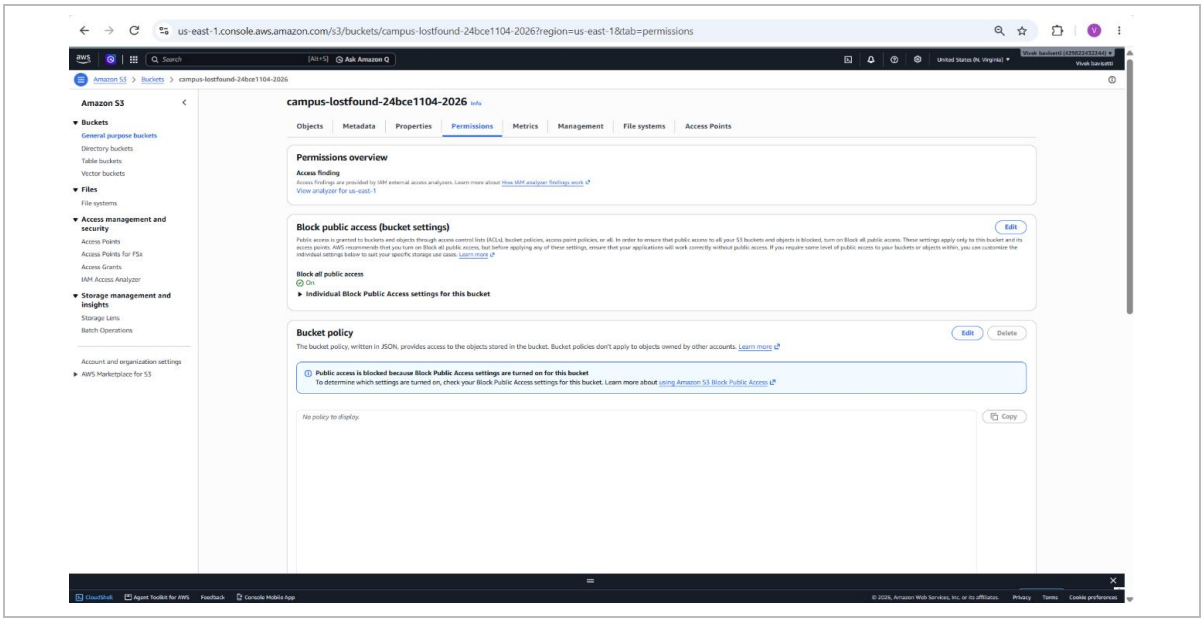


Figure 8.4 — Amazon S3 Console: bucket overview showing the items/ prefix and Block Public Access configuration

8.5 Amazon S3 – Uploaded Item Image

This screenshot shows an actual item photograph stored in the items/ prefix of the S3 bucket following a test report submission through the application. The object key follows the pattern items/<item-uuid>.<extension>, ensuring global uniqueness and traceability back to the corresponding DynamoDB record. The object's metadata panel confirms the content type (image/jpeg or image/png) and verifies that AES-256 server-side encryption was applied at upload time by the Flask boto3 call.

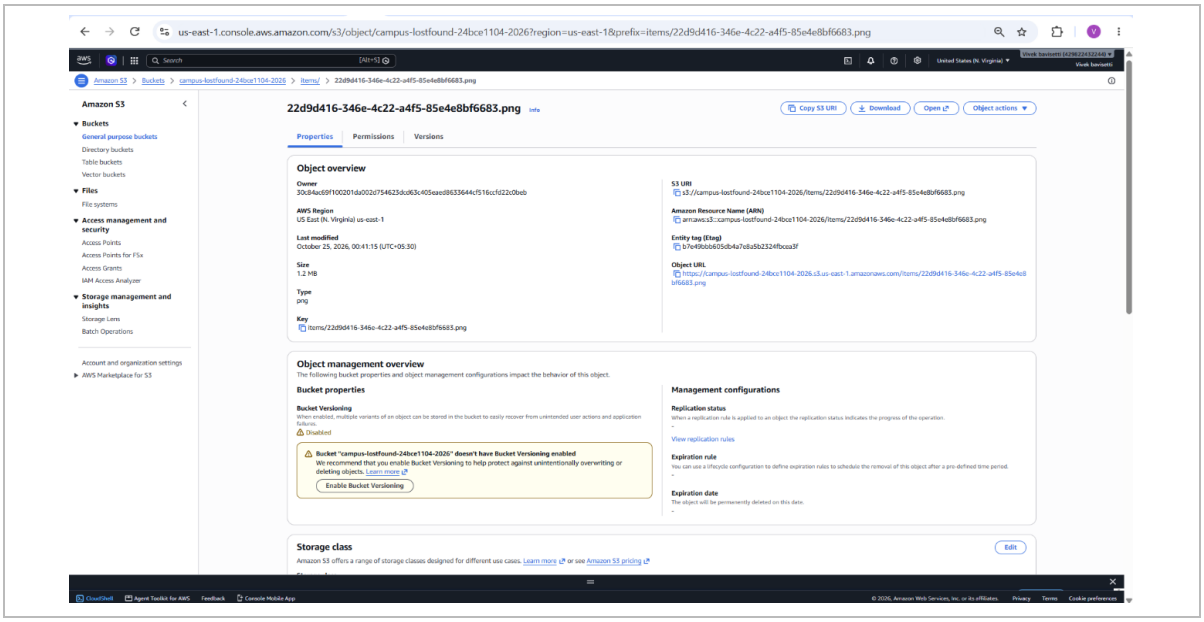


Figure 8.5 — Amazon S3 items/ prefix: uploaded item photograph with content type and AES-256 encryption confirmed

8.6 Amazon S3 – Encryption and Security Settings

The Properties tab of the S3 bucket confirms that SSE-S3 (Amazon S3-Managed Keys, AES-256 algorithm) is configured as the default encryption for all new objects written to the bucket. The Block Public Access settings panel shows all four options enabled: BlockPublicAcls, IgnorePublicAcls, BlockPublicPolicy, and RestrictPublicBuckets. Together these settings ensure that student data and photographs stored in the bucket cannot be accidentally or intentionally exposed to the public internet.

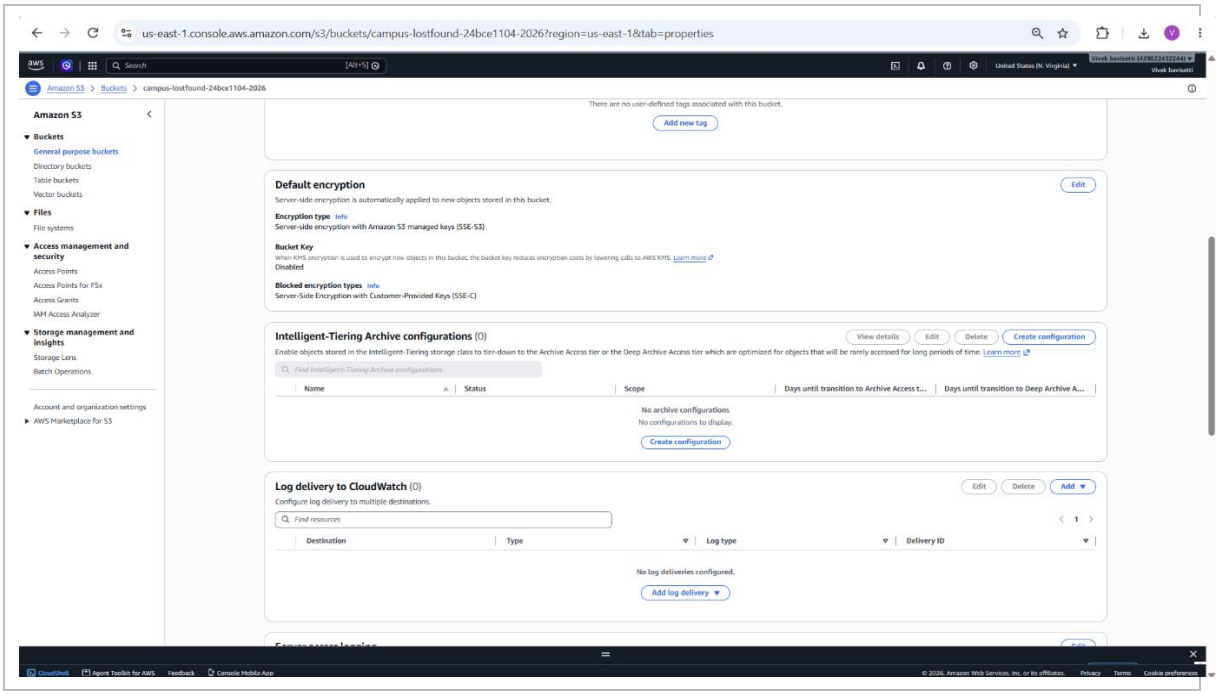


Figure 8.6 — S3 bucket Properties: AES-256 default encryption and all four Block Public Access options enabled

8.7 Amazon DynamoDB – Table Configuration

The DynamoDB table CampusLostFoundItems stores all item records created through the application. The table uses a simple primary key composed solely of the partition key item_id (String type) with no sort key, matching the UUID-based item identification used by Flask. The billing mode is PAY_PER_REQUEST (on-demand), which provides automatic scaling with zero provisioned-capacity charges — well suited to the variable and unpredictable usage patterns of a student-facing campus application.

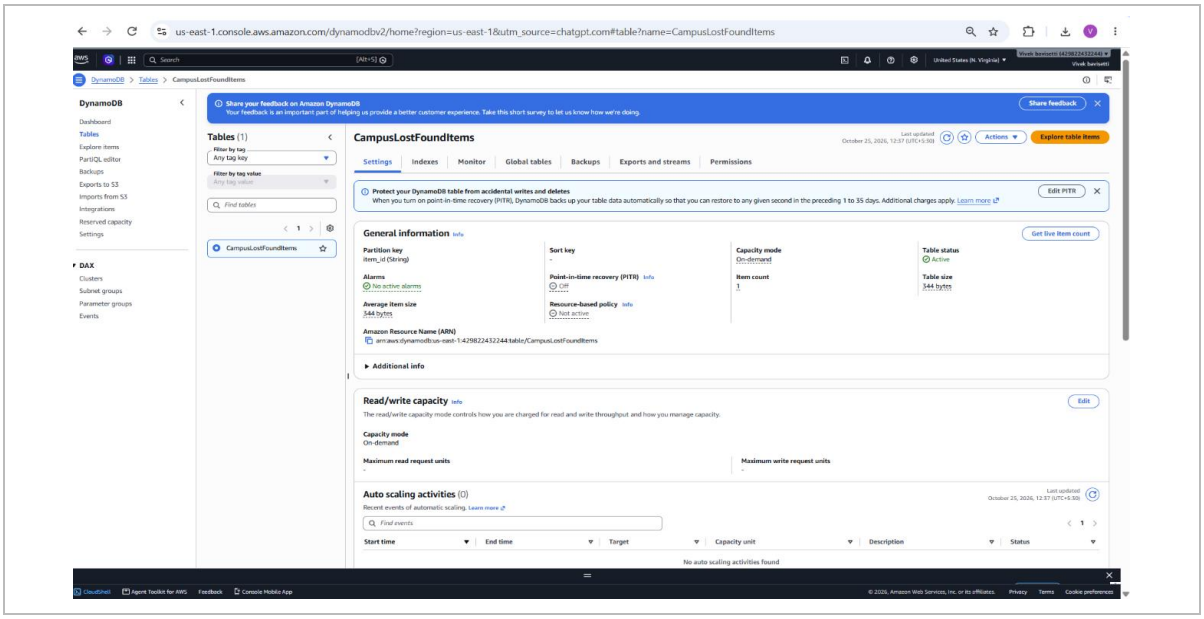


Figure 8.7 — DynamoDB Console: CampusLostFoundItems table — item_id PK (String), PAY_PER_REQUEST billing

8.8 Amazon DynamoDB – Item Record

A live item record viewed through the DynamoDB Explore Items interface confirms that the Flask application successfully writes all required attributes to the table: item_id (UUID, PK), item_type, title, description, location, contact, category, reporter, s3_key (the S3 object key of the uploaded photograph), status, created_at, and updated_at. This record is retrieved by the detail page using a GetItem call with item_id as the lookup key.

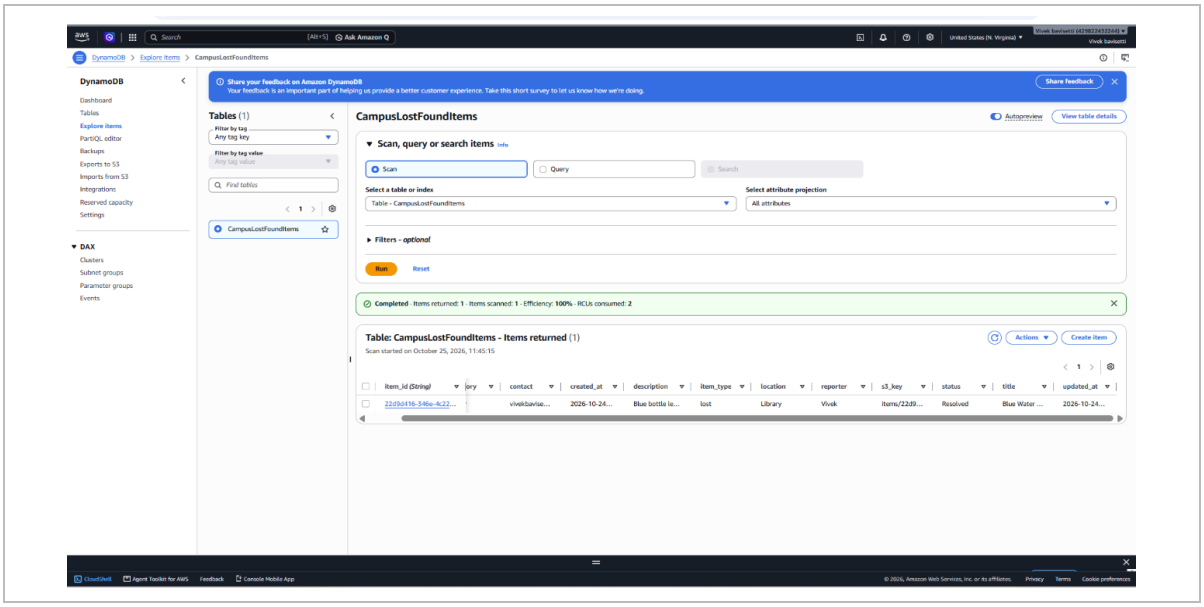


Figure 8.8 — DynamoDB Explore Items: a real item record showing all attributes written by the Flask application

8.9 IAM Role and Permissions

The IAM role DA3CampusLostFoundEC2Role is attached to the EC2 instance as an instance profile. It carries the policy DA3CampusLostFoundResourceAccess, which grants the minimum permissions required: S3 (GetObject, PutObject, HeadBucket, ListBucket) scoped to the project bucket; DynamoDB (PutItem, GetItem, UpdateItem, Scan, DescribeTable) scoped to the project table; and CloudWatch Logs (CreateLogGroup, CreateLogStream, PutLogEvents) scoped to the application log group. No other AWS actions are permitted.

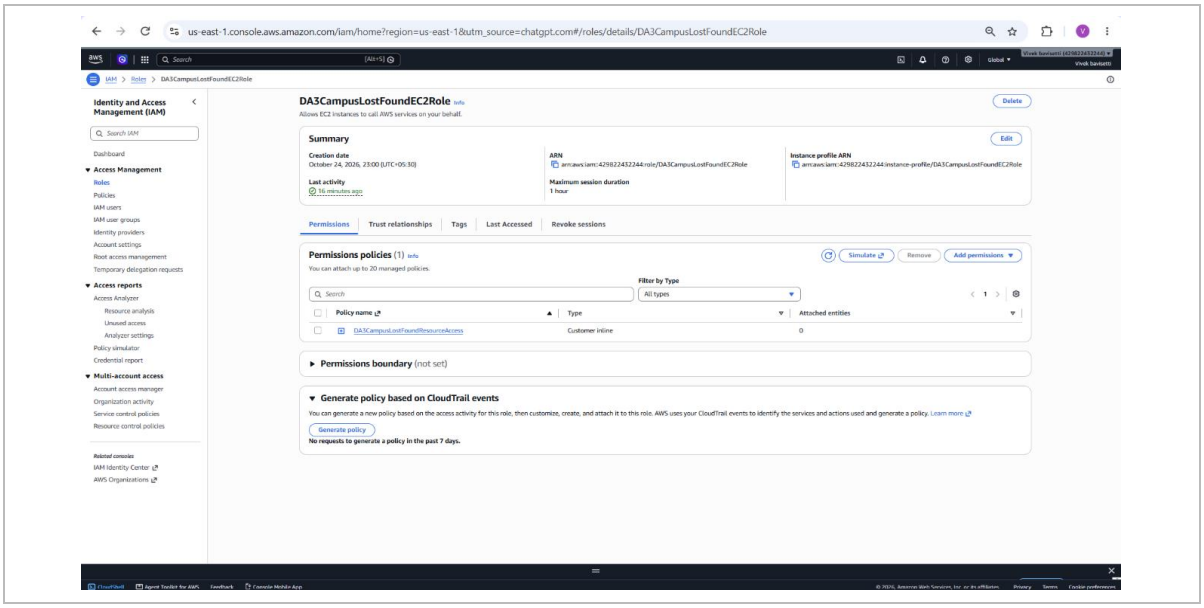


Figure 8.9 — IAM Console: DA3CampusLostFoundEC2Role with S3, DynamoDB, and CloudWatch Logs permissions

8.10 IAM Role – Trust Relationship

The trust relationship of DA3CampusLostFoundEC2Role specifies that only the EC2 service (ec2.amazonaws.com) can assume this role. This ensures the role cannot be assumed by any other AWS service or IAM user. At instance start, AWS automatically delivers temporary short-lived credentials to the EC2 instance metadata service; boto3 retrieves these transparently on each API call, so no access keys are ever written to the server's file system.

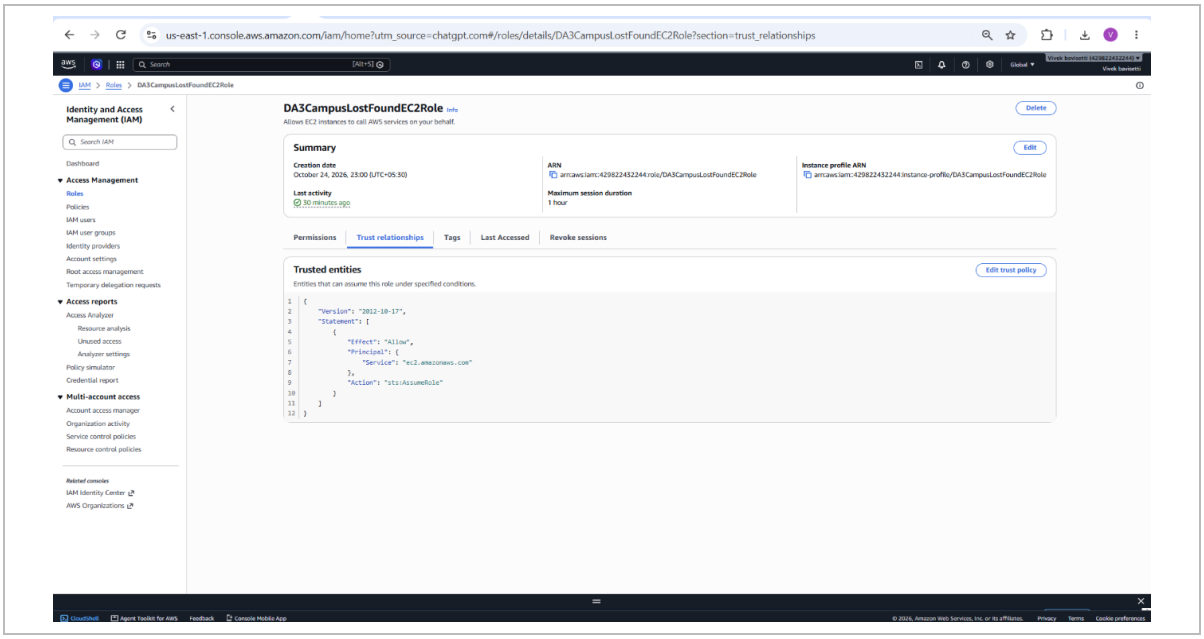


Figure 8.10 — IAM Trust Relationship: principal restricted to ec2.amazonaws.com only

8.11 Amazon CloudWatch – Logging Configuration

Amazon CloudWatch Logs captures runtime events emitted by the Flask application in the log group /campus-laf/app. Events are written each time a page loads, a report is submitted, an item status changes, or a health check runs. Each event carries a UTC millisecond timestamp and an INFO or ERROR severity label. The log group is configured with a seven-day retention policy, keeping storage costs negligible while retaining sufficient history for debugging.

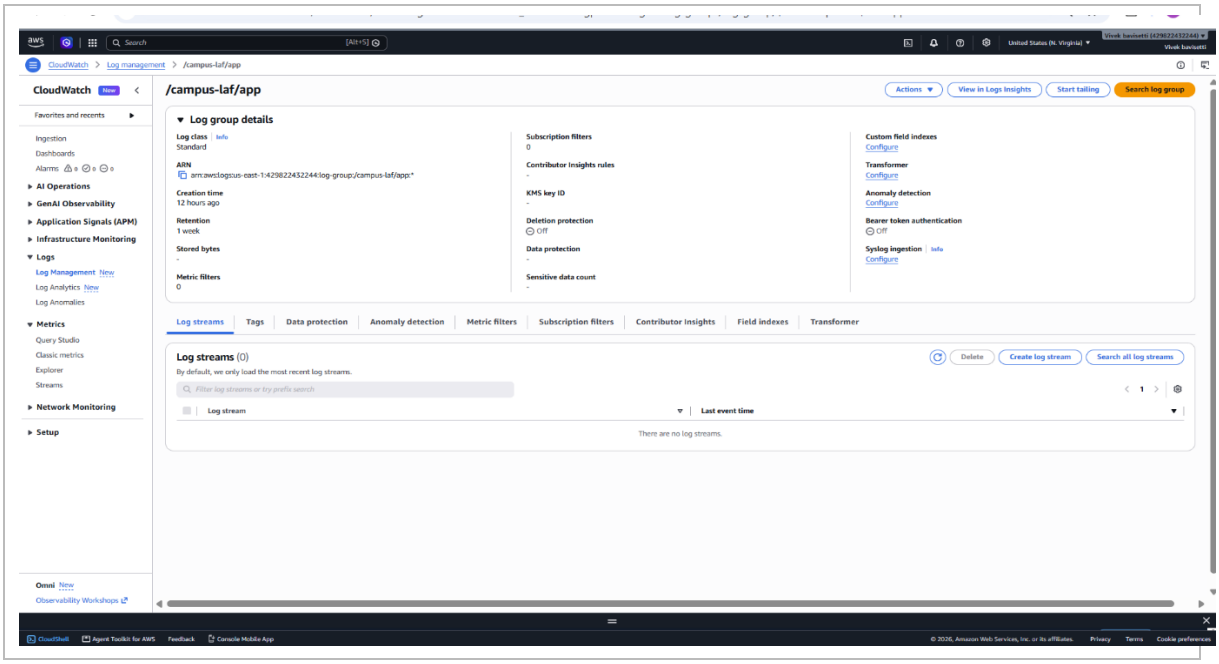


Figure 8.11 — CloudWatch Logs: /campus-laf/app log group with log streams and application events

8.12 Amazon CloudWatch – Metrics and Monitoring

CloudWatch automatically collects EC2 system metrics — CPU utilisation, network in/out, disk read/write operations — at one-minute granularity. The metrics view above shows stable, low CPU usage (well below 5%) during the test period, confirming that the t3.micro instance is adequately provisioned for the expected student traffic. In a production scenario these metrics can be used to trigger Auto Scaling policies or billing alarms.

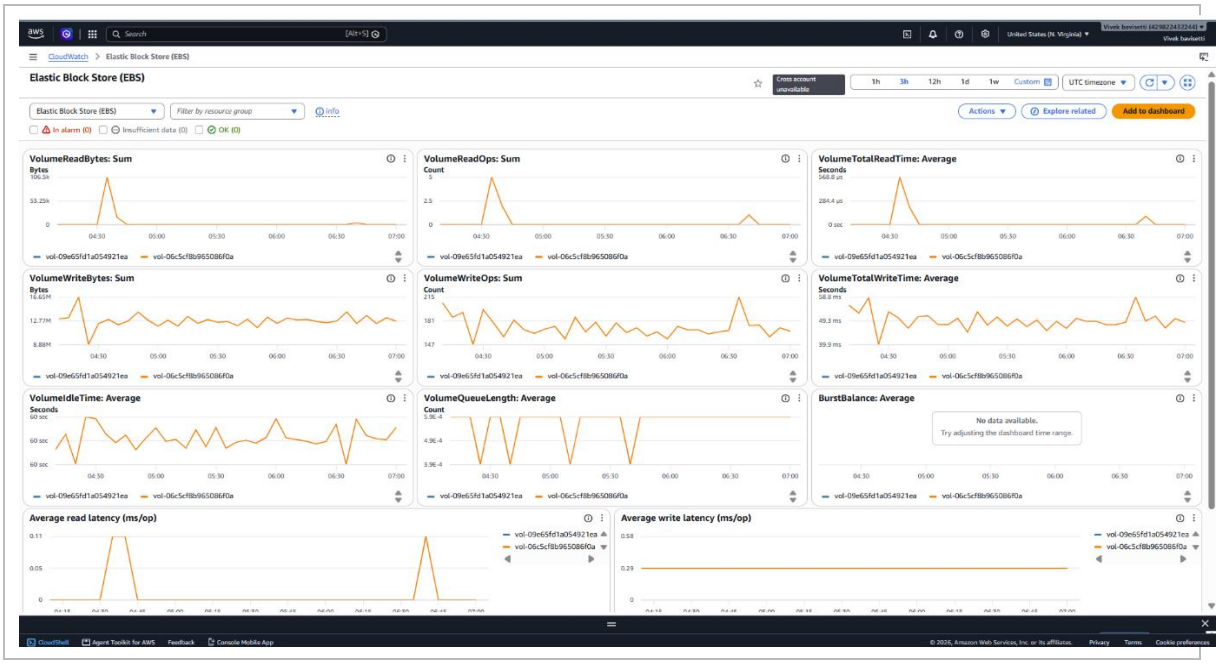


Figure 8.12 — CloudWatch Metrics: EC2 CPU utilisation and network metrics for the application server

9. Test Cases and Results

The following test cases were executed manually against the fully deployed application to verify end-to-end functionality across all features and AWS integrations. All twelve test cases passed without error.

#	Test Case	Expected	Actual	Result
TC-01	Submit lost item report with valid fields and a JPG image	Item saved in DynamoDB; image uploaded to S3; redirected to detail page	DynamoDB record visible; S3 object in items/ prefix; detail page loads correctly	PASS
TC-02	Submit found item report without an image	Item saved in DynamoDB with empty s3_key; "No photo" placeholder shown	Record written correctly; no broken image tag rendered on detail page	PASS
TC-03	Upload image larger than 5 MB	JS rejects file before form submission; error message shown to user	Error displayed; form not submitted; no S3 upload or DynamoDB write attempted	PASS
TC-04	Upload file with invalid extension (.pdf)	Client-side and server-side validation both reject; flash error displayed	Flash error shown; no record saved to DynamoDB; no S3 write	PASS
TC-05	Search by keyword matching item title	Only items whose title, description, or location contains the keyword are returned	Correct DynamoDB records returned; non-matching items excluded from results	PASS
TC-06	Filter results by category "Electronics"	Only items with category = Electronics are displayed on search results page	Filtered list matches expected DynamoDB Scan output; other categories excluded	PASS
TC-07	Filter results by status "Resolved"	Only resolved items are displayed	Correct items shown; open, claimed, and expired items excluded	PASS
TC-08	Update item status to "Claimed"	DynamoDB UpdateItem changes status; page reloads with amber Claimed badge	DynamoDB record status = Claimed; updated_at refreshed; badge colour updated	PASS
TC-09	Update item status to "Resolved"	Status updated; updated_at refreshed; CloudWatch event written	Green Resolved badge shown; CW log event confirmed in /campus-laf/app	PASS
TC-10	Access /api/health endpoint	JSON: dynamodb: ok, s3: ok, cloudwatch: ok; HTTP 200 returned	All three AWS service checks returned ok; 200 response received in browser	PASS
TC-11	Access item detail with invalid UUID in URL	Application returns HTTP 404 with custom error page; no unhandled exception	404 page rendered correctly; Python exception suppressed by error handler	PASS
TC-12	Verify S3 image URL is presigned, not public	Image URL in rendered HTML contains X-Amz-Expires= parameter; direct S3 URL returns HTTP 403	Presigned URL confirmed in page source; bucket remains private; 403 on direct access	PASS

10. Conclusion

The Campus Lost-and-Found System successfully demonstrates the design, development, and deployment of a full-stack, cloud-native web application on Amazon Web Services. All seven project objectives were met: the Flask application is publicly accessible via EC2; item photographs are stored securely in an encrypted, private S3 bucket and served through presigned URLs; all item records are persisted in a scalable DynamoDB table; IAM instance profile-based authentication eliminates stored credentials; and CloudWatch Logs provide an auditable event trail for all application activity.

Several important cloud engineering principles were reinforced through this project. Security by default was achieved by keeping the S3 bucket entirely private and serving images only through one-hour presigned URLs, so sensitive student data cannot be accessed without a valid application-generated token. Credential hygiene was maintained by using an IAM instance profile instead of access keys, removing the most common form of AWS credential exposure. Separation of concerns between the application tier (EC2), storage tier (S3), and database tier (DynamoDB) ensures that each component can be scaled or replaced independently.

Observability was established through CloudWatch Logs without requiring any third-party monitoring infrastructure, and cost consciousness was maintained throughout by choosing services — t3.micro EC2, S3 with SSE-S3, DynamoDB PAY_PER_REQUEST, and CloudWatch Logs with a 7-day retention — that fall within or close to the AWS Free Tier, making the project viable for a student budget with negligible ongoing expense.

Future enhancements could include: user authentication via Amazon Cognito so that only verified VIT students can post reports; email or SMS notifications through Amazon SES or SNS when a matching lost item is found; an Application Load Balancer with Auto Scaling groups to handle peak loads around semester examination periods; Amazon CloudFront as a CDN for static assets to reduce page-load latency across campus; and a mobile application built with React Native that consumes the existing Flask REST API. These extensions would build naturally on the current architecture without requiring a fundamental redesign, which itself demonstrates the scalability advantage of the AWS cloud-native approach taken in this project.

11. References

[1] Amazon Web Services, "Amazon EC2 User Guide for Linux Instances," AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/ec2/

[2] Amazon Web Services, "Amazon S3 Developer Guide," AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/s3/

[3] Amazon Web Services, "Amazon DynamoDB Developer Guide," AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/dynamodb/

[4] Amazon Web Services, "AWS Identity and Access Management User Guide," AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/iam/

[5] Amazon Web Services, "Amazon CloudWatch User Guide," AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/cloudwatch/

[6] Pallets Projects, "Flask Documentation (3.x)," 2024. [Online]. Available: https://flask.palletsprojects.com/

[7] boto3 Contributors, "Boto3 Documentation — AWS SDK for Python," 2024. [Online]. Available: https://boto3.amazonaws.com/v1/documentation/api/latest/index.html

[8] Nginx, Inc., "Nginx Documentation," 2024. [Online]. Available: https://nginx.org/en/docs/

[9] B. Chesneau, "Gunicorn — Python WSGI HTTP Server for UNIX," 2024. [Online]. Available: https://gunicorn.org/

[10] Amazon Web Services, "Using IAM Roles for Amazon EC2," AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/iam-roles-for-amazon-ec2.html

[11] Amazon Web Services, "Protecting Data Using Server-Side Encryption with SSE-S3," AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingServerSideEncryption.html

[12] Python Software Foundation, "Python 3 Documentation," 2024. [Online]. Available: https://docs.python.org/3/